# CycloTrack India — Complete Work Plan

Assigned to: Dhanesh

Primary ownership: ML/data engineering, real cyclone/provider integration, prediction services, risk intelligence, emergency data services and frontend integration.

Goal: replace demo/stub behavior with a genuinely working, testable, integrated system. The uploaded specification explicitly identifies authentication, analysis/upload, detection, classification, intensity, trajectory, full analysis, history, cyclone data, map, models and reports as missing from the current main backend, and states that the ML directories are scaffolding without trained weights. ■filecite■turn0file0■L668-L695■

## Phase 1 — ML/data engineering ownership

- ■ Inventory available cyclone datasets and determine which datasets can legally and technically support detection, classification, intensity and trajectory tasks.
- ■ Define dataset schemas, preprocessing steps, train/validation/test split rules and reproducible data-version tracking.
- ■ Build the real inference pipeline for detection/classification/intensity/trajectory using the repository's intended ML structure.
- ■ Train or integrate validated models only where data and compute are available. Record model artifacts, versions and evaluation metrics.
- ■ Create deterministic sample inputs and expected output ranges for backend integration tests.
- ■ Document model limitations and uncertainty. Never expose fabricated confidence or accuracy numbers.

## Phase 2 — Real cyclone data services

- ■ Implement/complete the cyclone data provider layer used by the backend.
- ■ Normalize GDACS/live cyclone records into a stable internal schema with source, timestamp, coordinates, alert level and provider identifier.
- ■ Add validation for malformed provider responses and geographic filtering for the India/Bay of Bengal region.
- ■ Implement caching and last-known-good behavior for temporary provider outages.
- ■ Coordinate the database schema for cyclone observations and trajectories with Yuvraj before migrations are finalized.
- ■ Provide realistic seed data for development/testing without representing it as live data.

## Phase 3 — Prediction services

- ■ Connect real model inference to the API service layer through clean interfaces, e.g. detect(), classify(), predict_intensity(), predict_trajectory().
- ■ Ensure preprocessing and postprocessing are deterministic and versioned.
- ■ Return physically meaningful units and coordinate conventions consistently.
- ■ Provide uncertainty/error estimates only when supported by the model/evaluation process.
- ■ Implement model health/load checks so the API can distinguish 'model loaded', 'model unavailable' and 'inference failed'.
- ■ Create integration tests proving that an uploaded sample reaches the model and produces a persisted result.

## Phase 4 — Risk, surge & flood intelligence

- ■ Replace demo risk scoring with a documented, testable risk calculation or a validated model.
- ■ Define the inputs, weights/features, thresholds and limitations of the risk calculation.
- ■ Integrate rainfall, wind, storm-surge and vulnerability inputs from real sources/models where available.

- [ ] Implement landfall, storm-surge and flood prediction services only after their data/model assumptions are documented.
- [ ] Expose model/data provenance in the API response so the frontend can tell users what generated the result.
- [ ] Test edge cases such as missing weather data, invalid coordinates, extreme values and stale provider data.

## Phase 5 — Emergency/shelter/missing-person data services

- [ ] Replace demo shelter registry with a real persistent shelter table and admin/import workflow.
- [ ] Implement geospatial nearest-shelter queries using PostGIS and verify distance calculations.
- [ ] Persist emergency help requests and missing-person reports with proper status transitions such as received, acknowledged, assigned, resolved.
- [ ] Add role-based responder access instead of relying only on a shared X-Responder-Token.
- [ ] Protect sensitive fields and ensure public endpoints do not leak phone numbers, emails or detailed personal information.
- [ ] Add audit fields and timestamps for responder actions.

## Phase 6 — Frontend integration & verification

- [ ] Integrate the finalized API contracts into the frontend without hard-coded demo values.
- [ ] Update map rendering to consume real GeoJSON and cyclone trajectory data.
- [ ] Connect dashboard/analysis pages to real analysis status and prediction results.
- [ ] Connect shelter, emergency, missing-person and alert screens to persistent backend data.
- [ ] Implement clear UI states for loading, no data, provider unavailable, model unavailable and API errors.
- [ ] Perform a complete end-to-end test with the shared test dataset and report all defects to Yuvraj.

## Dhanesh handoff checklist for Yuvraj

- [ ] Provide model artifact/version information, inference input schema and output schema.
- [ ] Provide dataset/provider documentation and sample payloads.
- [ ] Provide a list of known model limitations and failure cases.
- [ ] Provide integration tests or reproducible commands that prove the ML/data services work independently.

## Definition of Done

■ No endpoint returns fabricated success data merely to satisfy the frontend.

■ Every endpoint has validation, predictable error responses, logging, and tests.

■ Database writes and reads are real and survive server restarts.

■ Frontend calls the real API and handles loading, empty, error, and success states.

■ Authentication/authorization is enforced on protected resources.

■ ML results clearly distinguish real inference from unavailable/not-trained models.

■ External data providers have timeout, retry, validation, caching/rate-limit protection, and a clear degraded mode.

■ Security-sensitive data such as phone numbers, emergency reports, tokens and uploaded files is protected.

■ All migrations, environment variables, seed data, test commands, and deployment steps are documented.

■ The work is demonstrated end-to-end with reproducible test cases.

Integration rule: coordinate through shared API contracts and database schemas. Do not independently redesign routes already consumed by the frontend. The uploaded specification lists the target route architecture and recommends separating routers, services, PostgreSQL/PostGIS, and ML inference rather than keeping everything in one main.py. ■filecite■turn0file0■L1310-L1419■